

Object Oriented Programming

Contents

The model

1. [What object orientation is for](#)
2. [Encapsulation](#)
3. [Abstraction](#)
4. [Inheritance](#)
5. [Polymorphism](#)

Object semantics

6. [Identity and equality](#)
7. [equals and hashCode](#)
8. [Immutability](#)
9. [Records](#)

Building types

10. [Abstract classes and interfaces](#)
11. [Sealed types](#)
12. [Static members](#)
13. [Constructors and initialization](#)

Dispatch

14. [Overloading and overriding](#)
15. [Generics and variance](#)

Designing with objects

16. [Composition and delegation](#)
17. [Value objects and entities](#)
18. [Rich and anemic models](#)
19. [Absence and Optional](#)
20. [Exceptions in object design](#)

Reference

21. [Interview questions](#)

1. What object orientation is for

The usual answer, "it models the real world", is the weakest one available and does not survive a follow-up question. Most useful objects correspond to nothing physical, and forcing a model to mirror reality produces worse designs rather than better ones.

Object orientation is a way of managing dependencies. It bundles state with the behaviour that operates on that state, so that a change to how something is represented does not spread through every part of the system that uses it. Everything else follows from that.

Two framings worth knowing, because interviewers occasionally ask where the term comes from:

- **The mainstream, class-centric view:** classes define types, objects hold state, and inheritance and polymorphism organise them. This is the model Java, C# and C++ present.
- **Alan Kay's original conception,** which was about **messaging, late binding and complete isolation of state**. Kay has said the important idea was messaging rather than classes, which is a useful corrective when a design becomes about type hierarchies rather than about what objects ask of each other.

The four pillars, covered in the next four sections, are how the mainstream view is taught. They are worth being precise about because "encapsulation means private fields" and "abstraction means interfaces" are the answers that mark somebody as having memorised a list.

Pillar	The actual point
Encapsulation	Hide the representation so it can change without breaking callers
Abstraction	Expose what something does, not how, so callers depend on less
Inheritance	Share a contract, and secondarily an implementation
Polymorphism	One call site, several behaviours, chosen by the receiver

This note covers the language mechanics. The design principles that govern their use, including SOLID and composition over inheritance, are in the design principles note, and the recurring solutions built from them are in the design patterns note.

2. Encapsulation

Encapsulation is hiding the representation, not hiding the fields. The distinction matters, because a class with private fields and a public getter and setter for each one has hidden nothing at all: callers still depend on exactly the same structure, and changing it still breaks them.

```

// Private fields, and no encapsulation. The representation is fully exposed.
public class Order {
    private List<Item> items;
    public List<Item> getItems() { return items; }           // caller can mutate our
state
    public void setItems(List<Item> items) { ... }
}

// Encapsulated. Callers ask for what they need; the representation is ours.
public class Order {
    private final List<Item> items = new ArrayList<>();

    public void add(Item item) {
        if (items.size() >= MAX_ITEMS) throw new OrderFullException();
        items.add(item);
    }

    public List<Item> items() {
        return List.copyOf(items);                          // they cannot mutate ours
    }

    public Money total() { ... }                             // behaviour, not raw data
}

```

Two things the second version buys that the first cannot:

- **Invariants hold.** The order can guarantee it never exceeds the item limit, because every path that adds an item goes through code that checks. In the first version any caller can replace the whole list.
- **The representation can change.** Moving from a `List` to a `Map` keyed by product breaks nothing, because nobody outside the class knows there was a list.

The related idea is **information hiding**, from David Parnas: a module should hide the decisions most likely to change. That is a stronger instruction than "make fields private", because it says *what* to hide, and it is covered further in the design principles note.

Java's access levels, from most to least restrictive:

Modifier	Visible to
<code>private</code>	The declaring class only
<code>(none)</code> , <code>package-private</code>	The same package
<code>protected</code>	The same package, plus subclasses in other packages, through inheritance rather than through arbitrary instances
<code>public</code>	Everywhere

Package-private is the underused one. It allows classes that genuinely collaborate to see each other without exposing them to the whole application, and it is the natural default for anything that is not part of a deliberate public surface.

3. Abstraction

Abstraction is deciding what a caller needs to know and removing the rest. Where encapsulation hides a representation, abstraction hides a whole implementation behind a simpler idea of what the thing does.

The practical test is what breaks when the implementation changes. A caller written against `PaymentGateway` continues working when `StripeGateway` is replaced; a caller written against `StripeGateway` does not.

```
// The abstraction states what the caller needs: a charge either succeeds or fails.
public interface PaymentGateway {
    PaymentResult charge(Money amount, Card card);
}
```

The failure mode, and the one worth being able to name in an interview, is a **leaky abstraction**: one whose details escape through the interface anyway. A `PaymentGateway` whose method throws `StripeApiException`, or returns a `StripeResponse`, has not abstracted anything. Callers now depend on Stripe through a type that promised they would not.

The other failure is **abstracting too early**. An interface with one implementation, invented because a second one might appear, is usually the wrong shape when the second one finally arrives, because it was designed from imagination rather than from two real cases. The design principles note covers this under YAGNI.

4. Inheritance

Inheritance declares that one type is a specialisation of another, inheriting its contract and usually its implementation. In Java a class extends exactly one class and may implement many interfaces.

```
public class SavingsAccount extends Account {
    @Override
    public void withdraw(Money amount) {
        if (balance().minus(amount).isLessThan(MINIMUM_BALANCE)) {
            throw new InsufficientFundsException();
        }
        super.withdraw(amount);
    }
}
```

Inheritance is the most over-used mechanism in object orientation, and being able to say why is worth more in an interview than being able to use it.

It creates a particularly strong form of coupling between two classes. A subclass depends not only on the superclass's public surface but on its protected members and, in practice, on how its methods call one another. That last dependency is invisible and unstated, which produces the **fragile base class problem**: a change to the superclass that breaks subclasses without changing anything they can see.

```
public class CountingList<E> extends ArrayList<E> {
    private int added = 0;

    @Override public boolean add(E e) { added++; return super.add(e); }

    @Override public boolean addAll(Collection<? extends E> c) {
        added += c.size();
        return super.addAll(c);          // if addAll internally calls add, we count
    }
}
```

Whether that class is correct depends on an implementation detail of `ArrayList` that its documentation need not promise and that a future version may change. The composition version in section 16 has no such dependency.

The substitution requirement is the other constraint: a subclass must be usable anywhere the superclass is, without callers needing to know. Strengthening a precondition, weakening a postcondition or throwing something new violates it. This is the Liskov Substitution Principle, covered in the design principles note.

The rule that follows: **inherit for a contract, compose for reuse**. Wanting to reuse code is not a reason to inherit; wanting a genuine subtype relationship is.

5. Polymorphism

One call site, several possible behaviours, selected by the object receiving the call. It is the mechanism that lets a caller work with a type without knowing which implementation it has.

Three forms, and naming them separately is worth marks because most candidates describe only the third:

Form	In Java	Resolved
Ad hoc , overloading	Several methods, same name, different parameters	Compile time, by declared type
Parametric , generics	<code>List<T></code> , one implementation over many types	Compile time, then erased
Subtype , overriding	A subclass replaces a superclass method	Run time, by actual type

Subtype polymorphism is what people mean by the word, and it works through **dynamic dispatch**: the JVM looks at the object's actual runtime class, not the declared type of the variable holding it.

```
Account account = new SavingsAccount(); // declared type Account
account.withdraw(amount);                // SavingsAccount.withdraw runs
```

That single fact is what makes the mechanism useful: it is what allows a new implementation to be introduced without touching any caller, which is the Open Closed Principle in the design principles note.

Section 14 covers what happens when overloading and overriding appear together, which is what most questions on this topic actually test.

6. Identity and equality

Three different questions that get conflated, and separating them is a common interview opener:

Question	In Java
Are these the same object?	<code>a == b</code>
Are these objects equivalent?	<code>a.equals(b)</code>
Do these represent the same thing over time?	An identifier we chose, such as an order number

`==` compares **references** for objects and **values** for primitives. Two separately constructed objects with identical contents are never `==`.

```
String a = new String("hello");
String b = new String("hello");

a == b           // false. Two objects
a.equals(b)      // true. Equivalent contents
```

The `String` case has a wrinkle worth knowing because it is asked directly: **string literals are interned**, so `"hello" == "hello"` is `true` because both refer to the same pooled object, while `new String("hello")` deliberately creates a distinct one. This is an artefact of the constant pool rather than a rule about equality, and the durable answer is that comparing objects with `==` is a mistake regardless of what the pool happens to do.

The third row is the one people miss. **Some objects have an identity that outlives their contents**: a customer whose name and address both change is still the same customer. Section 17 covers that distinction, which drives real design decisions.

7. equals and hashCode

The most reliably asked question in this area, and one with a precise answer.

The `equals` contract. For non-null references, `equals` must be:

Property	Meaning
Reflexive	<code>x.equals(x)</code> is true
Symmetric	<code>x.equals(y)</code> implies <code>y.equals(x)</code>
Transitive	<code>x.equals(y)</code> and <code>y.equals(z)</code> implies <code>x.equals(z)</code>
Consistent	Repeated calls return the same result while nothing changes
Null-safe	<code>x.equals(null)</code> is false, never an exception

The `hashCode` contract, and the direction of it is the part people get backwards:

Equal objects must have equal hash codes. Unequal objects may share a hash code.

The second half is not a defect; it is why collisions are handled at all. The first half is the one that breaks things.

Why they must be overridden together

A hash-based collection finds a bucket from `hashCode` and only then compares with `equals`. Override `equals` alone and two equal objects land in different buckets, so the collection never compares them.

```
Set<Point> points = new HashSet<>();
points.add(new Point(1, 2));
points.contains(new Point(1, 2));    // false, if hashCode was not overridden
```

The object is in the set. The lookup goes to the wrong bucket and reports it is not.

```
@Override
public boolean equals(Object o) {
    if (this == o) return true;           // cheap path
    if (!(o instanceof Point other)) return false; // handles null and wrong type
    return x == other.x && y == other.y;
}

@Override
public int hashCode() {
    return Objects.hash(x, y);           // same fields, same order
}
```

Three details worth carrying:

- **Use the same fields in both.** A `hashCode` over more or fewer fields than `equals` breaks the contract.
- **Do not use mutable fields.** An object mutated after insertion into a `HashSet` hashes to a bucket it is no longer in, and becomes unreachable in a collection that still contains it. This is a strong argument for the immutability in section 8.

- `instanceof` against `getClass()` is a design choice, not a right answer. `getClass()` makes a subclass instance never equal to a superclass one, which keeps symmetry easy to reason about. `instanceof` permits cross-type equality, which is sometimes what a design wants, and then symmetry and transitivity have to be preserved deliberately rather than assumed. For value-like types the question disappears entirely by making the class `final` or using a record, which is one reason to prefer them.

8. Immutability

An immutable object cannot change after construction. It is the single most effective way to remove a category of bugs rather than to manage them.

What it buys:

- **Thread safety with no coordination**, because state that cannot change after construction needs no synchronising. This is the largest practical benefit, and it depends on the immutability being deep: an object whose fields are final but reference mutable objects is not thread-safe, which is the point section 9 makes about records.
- **Safe as a key**. Per section 7, a mutable object used as a map key can become unreachable. An immutable one cannot.
- **No defensive copying by callers**, because handing out a reference is safe.
- **Easier reasoning**. A value assigned at construction is the value forever, so nothing needs tracing.

Building one in Java:

```
public final class Money {                                // 1. final: no subclass
    can add mutability
    private final Currency currency;                      // 2. all fields final
    private final BigDecimal amount;
    private final List<String> tags;

    public Money(Currency currency, BigDecimal amount, List<String> tags) {
        this.currency = currency;
        this.amount = amount;
        this.tags = List.copyOf(tags);                    // 3. copy mutable input
    }

    public List<String> tags() { return tags; }            // 4. safe: already
    unmodifiable

    public Money plus(Money other) {                       // 5. operations return
    new instances
        requireSameCurrency(other);
        return new Money(currency, amount.add(other.amount), tags);
    }
}
```

The two steps people miss are 3 and 4: **copy mutable state on the way in and on the way out**. A constructor that stores the caller's `List` directly leaves the caller holding a reference that can still mutate our state, and a getter that returns the internal list does the same in reverse.

`final` on a field means **the reference cannot be reassigned**, not that the object it points at is immutable. `private final List<String> tags` with a plain `ArrayList` behind it is still mutable, which is why the copy matters.

The cost is allocation: every change creates an object. That is usually irrelevant, and where it is not, the answer is a mutable builder producing an immutable result rather than abandoning immutability.

9. Records

A **record** is a transparent carrier for a fixed set of values, and it removes most of the ceremony in the previous two sections.

```
public record Point(int x, int y) { }
```

That declaration provides a canonical constructor, an accessor per component (`x()`, not `getX()`), and `equals`, `hashCode` and `toString` derived from all components in order. The contract in section 7 is satisfied by construction, which removes an entire class of hand-written mistakes.

Records are **implicitly final**, their fields are **private and final**, and they **cannot extend a class**, though they may implement interfaces.

Validation goes in a compact constructor, which runs before the fields are assigned:

```
public record Money(Currency currency, BigDecimal amount) {  
    public Money {                                // compact: no parameter list,  
no assignments                                     // ordinary methods are allowed  
        Objects.requireNonNull(currency);  
        if (amount.scale() > currency.getDefaultFractionDigits()) {  
            throw new IllegalArgumentException("too many decimal places");  
        }  
    }  
  
    public Money plus(Money other) {  
        return new Money(currency, amount.add(other.amount));  
    }  
}
```

A **record is shallowly immutable**, which is the caveat worth stating. A record component of type `List` holds a reference that can still be mutated by whoever passed it, so a defensive copy in the compact constructor is still required:

```

public record Order(String id, List<Item> items) {
    public Order {
        items = List.copyOf(items);           // reassigning the parameter is
allowed here
    }
}

```

When **not** to use one: when the type has identity rather than value semantics, per section 17, or when the representation should be hidden. A record is transparent by design, so making one for something whose fields are an implementation detail gives away exactly what encapsulation was protecting.

10. Abstract classes and interfaces

A perennial interview question whose answer changed when interfaces gained default methods, so an answer that only recites the old table sounds dated.

	Abstract class	Interface
Instance fields	Yes	No, only <code>public static final</code> constants
Constructors	Yes	No
How many per type	One	Many
Method bodies	Yes	Yes, via <code>default</code> and <code>static</code>
Access modifiers on members	All	<code>public</code> , plus <code>private</code> helpers for defaults
Constructor-time state	Can enforce invariants at construction	Cannot

The distinction that survives default methods is state and construction. An interface cannot own instance state and cannot participate in object construction, so it cannot establish an invariant over constructor-controlled state the way an abstract class can. It can still constrain behaviour, through the contract its methods declare and through default methods that derive results from that contract; what it cannot do is guarantee a field was set correctly once and cannot be corrupted afterwards.

```

// Abstract class: shared state, established once, protected from subclasses.
public abstract class Account {
    private final String id;                // subclasses cannot corrupt this
    private Money balance;

    protected Account(String id, Money opening) {
        this.id = Objects.requireNonNull(id);
        this.balance = opening;
    }

    public abstract Money withdrawalLimit();    // subclasses supply the varying
part
}

// Interface: a contract, plus behaviour derived only from the contract.
public interface Discountable {
    Money price();
    Percentage discountRate();

    default Money discountedPrice() {          // derived, holds no state
        return price().minus(price().times(discountRate()));
    }
}

```

The practical guidance: **an interface expresses what a type can do, an abstract class expresses what a family of types shares**. Reach for the interface by default, because a class can implement many and extend only one, and introduce the abstract class when there is genuine shared state to protect.

Default methods exist mainly for evolution. They were added so that an interface could gain a method without breaking every existing implementation, which is why `Collection` could acquire `stream()` and `removeIf()`. Using them as a general code-sharing mechanism is a secondary use and produces the same coupling as inheritance without the ability to hold state.

Conflicting defaults are Java's version of the diamond problem, and it is resolved by rules rather than by silence: a class wins over an interface, a more specific interface wins over a less specific one, and where neither applies the compiler requires the implementing class to override and choose, optionally calling `Interface.super.method()`.

11. Sealed types

A **sealed** type controls which types may extend or implement it, which turns an open hierarchy into a closed set the compiler knows about.

```
public sealed interface Shape permits Circle, Square, Triangle { }

public record Circle(double radius) implements Shape { }
public record Square(double side) implements Shape { }
public record Triangle(double base, double height) implements Shape { }
```

Every permitted subtype must itself be `final`, `sealed`, or explicitly `non-sealed` to reopen the hierarchy. The declaration is the point: the set of possibilities is now stated and enforced.

What that buys is **exhaustiveness**. The compiler knows the complete set, so a `switch` covering every case needs no default and, more usefully, **fails to compile when a new case is added**:

```
double area = switch (shape) {
    case Circle c    -> Math.PI * c.radius() * c.radius();
    case Square s    -> s.side() * s.side();
    case Triangle t  -> t.base() * t.height() / 2;
};
// no default needed; adding a Shape breaks this
deliberately
```

That last property is the argument for sealed types, and it is worth stating precisely rather than as "the compiler catches missing cases", which overstates it. With an **open** hierarchy the compiler cannot know every subtype, so a switch has to carry a `default`, and that default silently absorbs any case added later. With a **sealed** hierarchy the compiler knows the permitted set, can verify exhaustiveness without a default, and therefore has something to report when the set grows.

The shift is from a default that quietly swallows new cases to a compile error that names every switch needing attention.

Sealed types and polymorphism solve the same problem from opposite directions, and knowing when to use which is the interesting part:

	Polymorphism	Sealed type with a switch
Adding a type	Easy. Nothing else changes	Every switch must be updated
Adding an operation	Every subclass must be changed	Easy. One new switch
Suits	An open set of types, a stable set of operations	A closed set of types, a growing set of operations

So a payment method with many implementations and one `charge` operation wants polymorphism. An expression tree with three node kinds and a dozen operations over them wants a sealed hierarchy. This trade is the "expression problem", and recognising which side a design sits on is a better answer than defaulting to either.

12. Static members

static means the member belongs to the class rather than to any instance, so it exists independently of how many objects are created and is reached through the class itself.

```
public class Order {
    private static final Money FREE_DELIVERY_THRESHOLD = Money.of("50.00");    //
    fine: constant
    private static int orderCount = 0;                                         //
    problem: shared state
}
```

Static **constants** are unremarkable. Static **state** is where the difficulties live, and being able to list them is the useful part:

- **Untestable.** Static state survives between tests, so one test's changes leak into another and the suite becomes order-dependent. The testing note treats this as a common source of flakiness.
- **A hidden dependency.** A class using a static field depends on it without saying so in its constructor, so the dependency is invisible to callers and cannot be substituted.
- **A concurrency hazard.** Shared mutable state accessible from everywhere, with no coordination implied by anything in the signature.

Two mechanical points that appear as interview questions:

- **Static methods are not polymorphic.** They are dispatched on the *declared* type at compile time. A subclass declaring a static method with the same signature **hides** it rather than overriding it, and calling it through a superclass-typed variable runs the superclass version. Section 14 shows this.
- **Static methods have no implicit `this`**, so they cannot reach instance members of the class they are declared in. They can of course use an instance handed to them as a parameter. This is what makes them suitable for genuinely stateless helpers, and a poor home for anything else.

The design guidance: **static is right for pure functions of their arguments** and for constants. Anything holding mutable state is better as an injected instance, where it can be substituted, scoped and tested.

13. Constructors and initialization

A constructor's job is to produce a valid object, and an object that can exist in an invalid state is a source of defects everywhere it is used.

```
public class Order {  
    private final String id;  
    private final Customer customer;  
  
    public Order(String id, Customer customer) {  
        this.id = Objects.requireNonNull(id, "id");  
        this.customer = Objects.requireNonNull(customer, "customer");  
    }  
}
```

Failing in the constructor is better than allowing a half-built object, which is the Fail Fast principle from the design principles note.

Initialization order is asked directly, and the sequence is:

Class initialization, once, the first time the class is needed

1. The superclass is initialized first
2. This class's static fields and static blocks, in source order

Object construction, on every new instance

3. The superclass is fully constructed: its fields, then its constructor body
4. This class's instance fields and initializer blocks, in source order
5. This class's constructor body

Both halves run superclass before subclass, and step 3 completing before step 4 is what produces the problem below.

The consequence that produces bugs:

Never call an overridable method from a constructor

The superclass constructor runs before subclass fields are assigned. If it calls a method the subclass overrides, the override executes against fields that are still at their default values.

```
class Base {
    Base() { init(); }                // runs first
    void init() { }
}
class Derived extends Base {
    private final String name = "set"; // assigned AFTER Base() completes
    @Override void init() {
        System.out.println(name);    // prints null
    }
}
```

Even the `final` field is null at that moment. From a constructor, call only methods that cannot dispatch to a subclass override, so `private`, `static` or `final`. That is necessary and not quite sufficient: a `final` method can itself call an overridable one, and the override is reached just the same. The check is the whole call chain, not the modifier on the first method in it.

Two related points:

- **this must not escape during construction.** Registering an object as a listener in its own constructor publishes a reference to something not yet fully built, which other threads may then use.
- **Telescoping constructors are a signal.** Four overloads differing by parameter count means the type has optional parts, and a builder states which values are being supplied instead of relying on position. The design patterns note covers the builder.

14. Overloading and overriding

Two words that sound alike, describe unrelated mechanisms, and are asked together for exactly that reason.

	Overloading	Overriding
What varies	The parameter list	The implementation
Where	Usually one class	A subclass
Resolved	Compile time , from the declared type	Run time , from the actual object
Return type	May differ freely	Must be the same or covariant
Access	May differ freely	Cannot be more restrictive
Also called	Static dispatch, early binding	Dynamic dispatch, late binding

The difference in resolution time is the whole question, and this is the example that shows it:

```
class Base { void whoAmI() { System.out.println("Base"); } }
class Derived extends Base {
    @Override void whoAmI() { System.out.println("Derived"); }
}

static void describe(Base b) { System.out.println("Base overload"); }
static void describe(Derived d) { System.out.println("Derived overload"); }

Base value = new Derived();

value.whoAmI();           // "Derived"           -> overriding: chosen by the ACTUAL type
describe(value);         // "Base overload" -> overloading: chosen by the DECLARED type
```

Both lines involve the same object. The method call follows the object; the overload follows the variable.

Four details that appear as follow-ups:

- **Return type alone cannot distinguish an overload.** Two methods with the same name and the same parameter list, differing only in what they return, do not compile. The signature that overload resolution uses is the name and the parameter types, and nothing else.
- **Covariant return types** are permitted on an *override*: it may return a subtype of what the superclass returned. This is what lets `clone()` return the specific type, and it is not a contradiction of the point above, which is about overloading.
- **An override cannot broaden what it throws.** It may throw fewer checked exceptions, or subtypes of the declared ones, and not a new checked exception the superclass did not declare. Unchecked exceptions are unconstrained.
- **Static methods are hidden, not overridden**, per section 12. A static method call resolves on the declared type, so `Base.whoAmI()` runs even when the variable holds a `Derived`. `@Override` on a static method is a compile error, which is one of several reasons to always write the annotation.
- **@Override is not optional in practice.** Without it, a typo in the method name or a wrong parameter type silently produces an unrelated new method, and the intended override never runs.

15. Generics and variance

Generics give **parametric polymorphism**: one implementation working over many types with compile-time checking.

Type erasure is the mechanism, and it explains most of the surprising restrictions. Generic type information is checked at compile time and largely removed afterwards, so at run time a `List<String>` is a `List`. Hence: no `new T[]`, no `instanceof List<String>`, and no overloading on `List<String>` against `List<Integer>`, since after erasure both are `List`.

Variance is the harder half, and the one interviews use to separate people.

```
Object[] objects = new String[1];           // compiles: arrays are covariant
objects[0] = 42;                             // ArrayStoreException at run time

List<Object> list = new ArrayList<String>(); // does NOT compile: generics are
invariant
```

Array covariance is a well-known type-safety trade-off: it was chosen early, before generics existed, so that code could work over arrays of any element type, and the cost is that an invalid assignment compiles and fails at run time with `ArrayStoreException`. **Generics are invariant** precisely to avoid repeating that, so `List<String>` is not a `List<Object>` even though `String` is an `Object`.

Wildcards reintroduce the flexibility safely, and the mnemonic for choosing is **PECS: Producer Extends, Consumer Super**.

```
// Producer: we only READ Ts out of it, so any subtype of T will do.
void copyFrom(List<? extends Number> source) {
    for (Number n : source) { ... }
    // source.add(1); will not compile: we do not know which subtype it holds
}

// Consumer: we only WRITE Ts into it, so any supertype of T will do.
void copyInto(List<? super Integer> target) {
    target.add(1);
    // Integer i = target.get(0); will not compile: we only know it holds some
    supertype
}
```

The reason is worth stating rather than the mnemonic alone: `? extends Number` could be a `List<Integer>` or a `List<Double>`, so adding a `Number` is unsafe, and reading one is always fine. `? super Integer` could be `List<Integer>` or `List<Object>`, so adding an `Integer` is always fine, and reading gives back only `Object`.

`Collections.copy(List<? super T> dest, List<? extends T> src)` is the canonical signature: the source produces, the destination consumes.

16. Composition and delegation

Composition builds behaviour by holding other objects rather than by extending them. The design principles note argues the preference; this section covers the mechanics of doing it.

The `CountingList` from section 4, rewritten to depend only on the public contract:

```

public class CountingList<E> implements List<E> {
    private final List<E> delegate;           // held, not extended
    private int added = 0;

    public CountingList(List<E> delegate) { this.delegate = delegate; }

    @Override public boolean add(E e) { added++; return delegate.add(e); }

    @Override public boolean addAll(Collection<? extends E> c) {
        added += c.size();
        return delegate.addAll(c);           // however addAll works internally, we
count once
    }

    // ... remaining List methods delegate straight through
}

```

The double-counting cannot occur, because this class never sees the delegate's internal calls. It depends on `List`'s published contract and nothing else, so a change inside `ArrayList` cannot break it.

What composition gives up is real: **it is more code**, since every method of the interface has to be forwarded, and it does not automatically expose new methods added to the delegate's concrete type. A new method on the interface may arrive as a default the wrapper inherits, and then it forwards to nothing and silently bypasses the wrapper's own behaviour, which is its own hazard.

What it gains:

- **Depends on the contract, not the implementation**, which is the whole of the fragile base class problem.
- **Can wrap any implementation**, including one chosen at run time. Inheritance fixes the implementation relationship structurally when the class is written, whereas a collaborator can be supplied and swapped later.
- **Can be composed in layers**, which is how the decorator pattern works.
- **Can hold several delegates**, where a class extends only one.

Interfaces plus composition also give a **has-a / can-do** split that inheritance conflates: a type *has* a `PaymentGateway` and *can do* `Discountable`. Modelling capabilities as interfaces and collaborators as fields keeps those separate.

Association, aggregation and composition are the vocabulary for how strongly one object holds another, and the words get used loosely enough that it is worth being able to separate them:

Relationship	Meaning
Association	Two objects know about each other or use each other. No ownership implied
Aggregation	A whole holds parts, and the parts exist independently. A team and its members: disband the team and the people remain

Relationship	Meaning
Composition	A whole owns its parts, and their lifetime is tied to it. An order and its line items: delete the order and they go with it

The distinction is about **lifetime and ownership**, not about the Java syntax, since all three are a field holding a reference. Where it shows up practically is in deciding what to copy, what to cascade on delete, and what may be shared between two owners.

17. Value objects and entities

Two kinds of object with different equality rules, and confusing them causes bugs that are hard to trace.

	Value object	Entity
Equality	All attributes equal	Identifiers equal
Identity	None beyond its value	An identifier that persists
Mutability	Immutable	Attributes change over time
Example	<code>Money</code> , <code>EmailAddress</code> , <code>DateRange</code>	<code>Customer</code> , <code>Order</code> , <code>Account</code>

Two £10 notes are interchangeable, so `Money` is a value. Two customers who happen to share a name and address are different people, so `Customer` is an entity, and remains the same customer after moving house.

That distinction decides `equals`:

```
public record Money(Currency currency, BigDecimal amount) { } // all fields,
automatically

public class Customer {
    private final CustomerId id;
    private String name; // changes, and is
not identity

    @Override public boolean equals(Object o) {
        if (this == o) return true;
        if (o == null || getClass() != o.getClass()) return false; // see below
        return id.equals(((Customer) o).id); // identity only
    }

    @Override public int hashCode() { return id.hashCode(); }
}
```

The two examples deliberately differ, which is the design choice from section 7 applied. The `Money` record compares by value and is implicitly final, so cross-type equality cannot arise. `Customer` is an ordinary class that may well be subclassed, and `getClass()` keeps the relation symmetric without anyone having to reason

about it. Using `instanceof` here is defensible, and then subclass equality has to be designed rather than assumed, so `getClass()` is the safer default for an entity whose hierarchy is open.

The habit worth forming is **making value objects out of what would otherwise be primitives**, sometimes called avoiding primitive obsession:

```
// Every parameter is a String. Swapping two of them compiles and is wrong.
void transfer(String fromAccount, String toAccount, String amount, String currency)

// The types say what they are, and the compiler enforces it.
void transfer(AccountId from, AccountId to, Money amount)
```

The wrapper also gives validation one home. An `EmailAddress` that cannot be constructed from an invalid string means nothing downstream needs to check, which the security note's input-validation section relies on.

18. Rich and anemic models

The distinction between objects that have behaviour and objects that only carry data.

```
// Anemic: a data holder. The behaviour lives elsewhere.
public class Order {
    private OrderStatus status;
    private List<Item> items;
    // getters and setters for everything
}

public class OrderService {
    public void ship(Order order) {
        if (order.getStatus() != PAID) throw new IllegalStateException();
        order.setStatus(SHIPPED); // rules live here, and in every
other caller
    }
}
```

```
// Rich: the object protects its own rules.
public class Order {
    private OrderStatus status;

    public void ship() {
        if (status != PAID) {
            throw new IllegalStateException("cannot ship an unpaid order");
        }
        status = SHIPPED;
    }
}
```

The complaint about the anemic version is not aesthetic. **The rules are separated from the data they govern**, so every place that changes status must remember the same checks, and nothing prevents a caller from setting the field directly. The invariant is a convention rather than a guarantee, which is the encapsulation failure from section 2 stated at the level of a whole design.

Martin Fowler named the anemic domain model as an anti-pattern for that reason. Two honest qualifications, because an interview answer that treats it as a universal error is easy to push back on:

- **Some types genuinely are data.** A request body, a response object, a row mapped from a database, a message being serialised: these carry data across a boundary and have no rules to protect. Records suit them.
- **The service layer does not disappear.** Behaviour spanning several objects, or requiring collaborators the object should not know about, belongs in a service. The test is whether the logic concerns one object's own invariants, in which case it belongs on that object.

19. Absence and Optional

`null` is available for every reference type and says nothing about intent, which is why an unexpected `NullPointerException` is the most common failure in Java code.

Optional makes absence part of the type. A method returning `Optional<Customer>` states in its signature that there may be no customer, and the caller cannot ignore it as easily as ignoring a possible null.

```
public Optional<Customer> findByEmail(String email) { ... }

String name = customers.findByEmail(email)
                        .map(Customer::name)
                        .orElse("unknown");
```

Optional was designed for return types, and using it elsewhere is the common misuse:

Use	Guidance
Return type of a method that may find nothing	What it is for. Prefer it
Field	Avoid as a habit. It is not serialisable, adds a wrapper per object, and a field can simply be null
Method parameter	Avoid as a habit. It forces every caller to wrap, and an overload usually reads better
Collection element	Avoid. An empty collection already expresses absence

Those are conventions rather than prohibitions. Occasional cases justify a field or a parameter, and the reason to treat them as exceptions is that doing it routinely complicates the object model and the API without buying much.

Two anti-patterns worth recognising. `optional.get()` without checking throws `NoSuchElementException` when empty, so it converts one unchecked failure into another while adding a wrapper; `orElseThrow` says the same thing deliberately, and the no-argument form is the clearer way to write it. And `if (optional.isPresent()) { optional.get() ... }` is the null check rewritten longhand, missing the point of `map`, `filter`, `orElse` and `ifPresent`.

Where `Optional` does not fit, the alternatives are to **return an empty collection rather than null** for anything list-shaped, and to use the **null object pattern**, an implementation that does nothing safely, where a caller genuinely has no reasonable action to take on absence.

20. Exceptions in object design

Exceptions are part of a type's contract, and treating them as an afterthought produces callers that cannot respond sensibly.

	Checked	Unchecked
Extends	<code>Exception</code>	<code>RuntimeException</code>
Declaration	Must be declared or handled	Neither required
Commonly used for	Conditions the caller is expected to consider	Programming errors and invalid state, where the caller is not required to handle anything

The bottom row is a convention rather than a rule, and treating it as a definition is easy to challenge.

`IllegalArgumentException` and `NumberFormatException` are unchecked and entirely recoverable, and plenty of checked exceptions arrive where the caller can do nothing useful. **What the language enforces is the declaration, not the recoverability.**

Java is the main language to make checked exceptions part of the type system, and the design has been argued about ever since. The position worth holding:

Use an exception for what the caller can act on, and make that action possible. An exception saying "payment declined" that the caller can retry differently is useful; one saying "something went wrong" is not.

Three practices:

- **Do not catch and swallow.** An empty `catch` block deletes information about a failure that already happened. If there is genuinely nothing to do, that deserves a comment explaining why.
- **Do not use exceptions for control flow.** They are expensive to construct because of the stack trace, and they hide the ordinary path from the reader. A `findById` returning `Optional` beats one throwing when a record is absent, because absence is not exceptional.

- **Preserve the cause.** Wrapping is fine and losing the original is not, so pass the cause into the new exception. The observability note covers what the log needs.

The one that connects to this whole document: **an exception should not leak the implementation**. A repository throwing `SQLException` through its interface forces every caller to know it is backed by a database, which is the leaky abstraction from section 3 appearing in the failure path rather than the success path.

21. Interview questions

As in the other notes, each question has an **opening** of roughly thirty to forty seconds, whose last sentence points at the deeper material, and a **follow-up** for when the interviewer takes it.

"What are the four pillars of object orientation?"

Encapsulation, abstraction, inheritance and polymorphism, though I would rather describe what each is for than define them, because the definitions are where people get caught. Encapsulation is hiding the representation so it can change without breaking callers. Abstraction is exposing what something does rather than how. Inheritance shares a contract, and secondarily an implementation. Polymorphism gives one call site several behaviours, chosen by the object receiving the call.

If they go deeper:

The thing I would push back on is "encapsulation means private fields", because a class with a getter and setter for every private field has hidden nothing: callers still depend on exactly the same structure and changing it still breaks them. Real encapsulation means the class exposes operations rather than data, so it can hold its invariants and change its representation freely. Underneath all four, what object orientation is actually managing is dependencies: bundling state with the behaviour over that state so a change to the representation does not spread. I would also avoid saying it models the real world, since most useful objects correspond to nothing physical.

"What is the difference between `==` and `equals` ?"

`==` compares references for objects and values for primitives, so it asks whether these are the same object. `equals` asks whether two objects are equivalent, and what that means is up to the class. Two separately constructed objects with identical contents are never `==` but should usually be `equals`. The string case is the one most often got wrong, because string literals are interned so `"a" == "a"` is true, while `new String("a") == new String("a")` is false.

If they go deeper:

That interning is an artefact of the constant pool rather than a rule about equality, and the durable answer is that comparing objects with `==` is a mistake regardless of what the pool does. There is also a third question people conflate with these two, which is whether two objects represent the same thing over time. A customer whose name and address both change is still the same customer, so its equality is by identifier rather than by contents. That is the value-object against entity distinction, and it decides how `equals` should be written: all attributes for a value like `Money`, identifier only for an entity like `Customer`.

"Why must `equals` and `hashCode` be overridden together?"

Because hash-based collections use both. A `HashMap` or `HashSet` finds a bucket from `hashCode` first, and only compares with `equals` within that bucket. So if we override `equals` and not `hashCode`, two objects that are equal get different hash codes, land in different buckets, and the collection never compares them. The visible symptom is adding an object to a set and then having `contains` return false for an equal one.

If they go deeper:

The contract is directional and people get it backwards: equal objects must have equal hash codes, while unequal objects may share one. The second half is not a defect, it is why collision handling exists. Two details I would mention: use exactly the same fields in both, and do not use mutable fields at all, because an object mutated after insertion hashes to a bucket it is no longer in and becomes unreachable in a collection that still contains it. That is a strong argument for making anything used as a key immutable, and it is one reason records are convenient, since they derive both methods from the components and cannot get the contract wrong.

"When would you use an abstract class rather than an interface?"

The distinction that still holds after default methods is state and construction. An interface cannot own instance state and cannot participate in construction, so it cannot establish an invariant over constructor-controlled state the way an abstract class can. My default is the interface, because a class can implement many and extend only one, and I reach for the abstract class when there is real shared state rather than just shared code.

If they go deeper:

To be precise about the invariant point: an interface can still constrain behaviour through its contract and through defaults derived from that contract. What it cannot do is guarantee a field was set correctly once and cannot be corrupted afterwards, which is what a constructor plus private final state gives us. It is also worth knowing why default methods were added, which was evolution rather than code sharing: they let an interface gain a method without breaking every existing implementation, which is how `Collection` acquired `stream` and `removeIf`. Using them as a general reuse mechanism gives the same coupling as inheritance without the ability to hold state. Conflicting defaults are Java's version of the diamond problem, and it is resolved by rules rather than left ambiguous: a class wins over an interface, a more specific interface wins over a less specific one, and where neither applies the compiler forces the implementing class to override and choose.

"What is the difference between overloading and overriding?"

Overloading is several methods with the same name and different parameter lists, and it is resolved at compile time from the declared type. Overriding is a subclass replacing a superclass method, and it is resolved at run time from the object's actual type. So the two mechanisms are unrelated despite the similar names, and the difference in when they resolve is the whole thing. The example that shows it is a variable declared as the supertype holding a subtype instance.

If they go deeper:

With `Base value = new Derived()`, calling an overridden method runs the `Derived` version because dispatch follows the object, while passing that same variable to overloaded `describe(Base)` and `describe(Derived)` selects the `Base` overload because overload resolution follows the variable. Same object, two different rules. Three follow-ons I would have ready: covariant return types are allowed, so an override may return a subtype; static methods are hidden rather than overridden, since they resolve on the declared type; and I always write `@Override`, because without it a typo in the name silently creates an unrelated new method and the intended override never runs.

"Why are generics invariant when arrays are covariant?"

Because array covariance was a mistake, and generics avoided repeating it. `Object[] a = new String[1]` compiles, and then assigning an integer into it throws `ArrayStoreException` at run time. That is a type error caught far too late. Generics are invariant, so `List<String>` is not a `List<Object>` even though `String` is an `Object`, and the equivalent mistake fails to compile instead.

If they go deeper:

The flexibility comes back through wildcards, and the mnemonic is PECS, producer extends, consumer super. If a parameter only produces values for us to read, `? extends T` is right, because any subtype will do and we can always read a `T` out. If it only consumes values we write into it, `? super T` is right, because any supertype can accept a `T`. The reason rather than the mnemonic: `List<? extends Number>` might be a `List<Integer>`, so adding a `Double` would be unsafe, and `List<? super Integer>` might be `List<Object>`, so reading gives back only `Object`. `Collections.copy(List<? super T> dest, List<? extends T> src)` is the canonical signature. Underneath all of it is erasure, which is also why there is no `new T[]` and no `instanceof List<String>`.

"What is the difference between association, aggregation and composition?"

They describe how strongly one object holds another, and the difference is about lifetime and ownership rather than about syntax, since all three are a field holding a reference. Association is the weakest: two objects know about or use each other with no ownership implied. Aggregation is a whole holding parts that exist independently, like a team and its members, where disbanding the team leaves the people. Composition is a whole that owns its parts, like an order and its line items, where deleting the order takes them with it.

If they go deeper:

Where it actually matters is in three decisions. What gets copied, since composed parts usually need copying with the whole and aggregated ones do not. What cascades on delete, which is the same distinction expressed in a database. And what may be shared between two owners, which composed parts must not be. The confusing part of the vocabulary is that "composition" here means ownership, while "composition over inheritance" means holding a collaborator rather than extending it, and those are different uses of the word. In practice I would say the ownership distinction is a modelling decision that the language does not enforce, so it has to live in how the code is written.

"A superclass constructor calls a method the subclass overrides. What happens?"

The override runs, and it runs against subclass fields that have not been assigned yet, so they are all at their default values. The order is that the superclass is fully constructed before any subclass field initializer executes, and dispatch is still dynamic during that window, so the subclass method is selected while the subclass object is only half built. A `final String` field that clearly has an initializer will read as null inside that call.

If they go deeper:

That is why the rule is to call only methods that cannot dispatch to a subclass override from a constructor, so `private`, `static` or `final`. I would add that this is necessary rather than sufficient, because a `final` method can itself call an overridable one and the override is reached just the same, so what matters is the whole call chain rather than the modifier on the first method. The related failure is letting `this` escape during construction, for example by registering the object as a listener in its own constructor, which publishes a reference to something not yet fully built and lets another thread observe it in that state. Both come from the same underlying fact, that an object exists and is reachable before it is finished. Where a design genuinely needs subclass input during construction, passing it as a constructor parameter or using a factory method that constructs and then initialises avoids the window entirely.

"What is wrong with inheritance?"

Nothing, when it expresses a genuine subtype relationship. The problem is that it creates a particularly strong form of coupling between two classes, and gets used for code reuse, which it is bad at. A subclass depends not only on the superclass's public surface but on how its methods call one another, and that dependency is invisible and unpromised, which is the fragile base class problem: a change inside the superclass breaks subclasses without changing anything they can see.

If they go deeper:

The classic demonstration is extending `ArrayList` to count additions, overriding both `add` and `addAll`. Whether it double-counts depends on whether `addAll` internally calls `add`, which is an implementation detail the documentation need not promise and a future version may change. The composition version holds a `List` and delegates, so it depends only on the published contract and cannot be broken that way. The other constraint is substitutability, that a subclass must be usable anywhere the superclass is, so strengthening a precondition or throwing something new violates it. The rule I follow is inherit for a contract, compose for reuse, and wanting to reuse code is never by itself a reason to inherit.

"How would you make a class immutable, and why bother?"

Make the class `final` so nobody can subclass it and add mutability, make every field `private final`, provide no setters, and copy any mutable state both on the way in and on the way out. Those last two are the steps people miss: a constructor storing the caller's `List` directly leaves them holding a reference into our state, and a getter returning the internal list does the same in reverse. The main reason to bother is that an immutable object is thread-safe with no coordination at all.

If they go deeper:

It is also safe as a map key, which matters because a mutable key mutated after insertion becomes unreachable in a collection that still contains it. And it removes a whole category of reasoning, since a value assigned at construction is the value forever. One precision point: `final` on a field means the reference cannot be reassigned, not that what it points at is immutable, so a `final` field holding an `ArrayList` is still mutable and that is exactly why the defensive copy matters. Records give most of this for free but are only shallowly immutable, so a record with a `List` component still needs `List.copyOf` in the compact constructor. The cost is allocation on every change, and where that genuinely matters the answer is a mutable builder producing an immutable result rather than giving up immutability.

"What is an anemic domain model?"

A design where objects carry data and nothing else, with all the behaviour in service classes that reach in through getters and setters. The complaint is not aesthetic: the rules end up separated from the data they govern, so every place that changes an order's status has to remember the same checks, and nothing stops a caller setting the field directly. The invariant becomes a convention rather than a guarantee, which is encapsulation failing at the level of a whole design.

If they go deeper:

The rich alternative puts the rule on the object, so `order.ship()` checks that it is paid rather than `OrderService` checking before calling a setter. Fowler named the anemic version as an anti-pattern for that reason. I would qualify it two ways, though. Some types genuinely are data: a request body, a response object, a row mapped from a database, a message being serialised, all of which carry data across a boundary and have no rules to protect, and records suit those well. And the service layer does not disappear, because behaviour spanning several objects, or needing collaborators the object should not know about, still belongs there. The test I use is whether the logic concerns one object's own invariants.

22. Common mistakes

- Answering that object orientation "models the real world", which does not survive a follow-up and leads to worse designs.
- Treating encapsulation as private fields, then adding a getter and setter for each, which hides nothing.
- Returning the internal collection from a getter, so callers can mutate state the class believes it controls.
- Storing a caller's mutable collection directly in a constructor, which leaves them holding a reference into our state.
- Assuming `final` on a field makes the referenced object immutable, when it only prevents reassignment.
- Overriding `equals` without `hashCode`, so equal objects land in different buckets and hash-based lookups fail.

- Using mutable fields in `equals` or `hashCode`, so an object mutated after insertion becomes unreachable in a collection that still contains it.
- Comparing objects with `==`, and concluding from interned string literals that it sometimes works.
- Giving an entity value-based equality, so two different customers with the same name are treated as one.
- Extending a class purely to reuse its code, which takes a particularly strong form of coupling for the weakest reason.
- Overriding methods of a class whose internal call structure is not part of its contract, which is the fragile base class problem.
- Calling an overridable method from a constructor, so the override runs against fields that are still null.
- Letting `this` escape during construction, publishing an object that is not yet fully built.
- Omitting `@Override`, so a typo silently creates a new method and the intended override never runs.
- Expecting a static method to be overridden, when it is hidden and resolves on the declared type.
- Holding mutable static state, which is a hidden dependency, a concurrency hazard, and a source of order-dependent tests.
- Using `Optional` as a field or a parameter, when it was designed for return types.
- Writing `if (opt.isPresent()) opt.get()`, which is a null check with extra steps.
- Returning `null` for a collection instead of an empty one.
- Catching an exception and swallowing it, deleting the information about a failure that already happened.
- Using exceptions for ordinary control flow, such as throwing when a record is simply absent.
- Letting an implementation exception such as `SQLException` escape through an interface, which leaks the abstraction on the failure path.
- Passing several `String` parameters where value objects would let the compiler catch a swapped argument.

23. Quick reference

Term	Meaning
Encapsulation	Hide the representation so it can change. Not "private fields with getters"
Abstraction	Expose what, not how. Broken when implementation types escape through the interface
Leaky abstraction	The implementation shows through anyway, such as <code>SQLException</code> on a repository interface
Inheritance	Shares a contract, and couples subclass to superclass strongly. Inherit for a contract, compose for reuse
Fragile base class	A superclass change breaks subclasses that depend on its internal call structure
Polymorphism	One call site, behaviour chosen by the receiving object

Term	Meaning
Ad hoc, parametric, subtype	Overloading, generics, overriding
Dynamic dispatch	Overriding resolves at run time from the actual type
Static dispatch	Overloading resolves at compile time from the declared type
<code>==</code> against <code>equals</code>	Same object, against equivalent objects
<code>equals</code> contract	Reflexive, symmetric, transitive, consistent, false for null
<code>hashCode</code> contract	Equal objects must share a hash. Unequal objects may
Method hiding	A static method in a subclass hides rather than overrides
Covariant return	An override may return a subtype
<code>@Override</code>	Not optional. Without it a typo becomes a new method silently
Immutable class	<code>final</code> class, <code>private final</code> fields, no setters, copy in and copy out
<code>final</code> field	The reference cannot be reassigned. The object may still be mutable
Record	Implicitly final, accessors, <code>equals</code> , <code>hashCode</code> , <code>toString</code> derived. Shallowly immutable
Compact constructor	Validation and normalisation before fields are assigned
Sealed type	A closed set of subtypes, giving compile-time exhaustive switches
Expression problem	Polymorphism suits new types; sealed plus switch suits new operations
Default method	For interface evolution, not general code sharing. Holds no state
Diamond resolution	Class beats interface, specific beats general, otherwise override and choose
Class initialization	The superclass first, then this class's static fields and blocks
Object construction	The superclass fully constructed, then this class's instance fields, then its constructor body
Value object	Equal by all attributes. Immutable. <code>Money</code> , <code>EmailAddress</code>
Entity	Equal by identifier. Attributes change. <code>Customer</code> , <code>Order</code>
Primitive obsession	Strings and ints where a value object would let the compiler catch mistakes
Anemic domain model	Data holders plus service classes holding the rules the data should protect
Type erasure	Generic types checked then largely removed. No <code>new T[]</code> , no <code>instanceof List<String></code>
Invariance	<code>List<String></code> is not a <code>List<Object></code> , deliberately, unlike arrays
PECS	Producer <code>extends</code> , consumer <code>super</code>

Situation	What to reach for
A class with a getter and setter per field	Ask what operations callers actually need, and expose those instead
A getter returning a mutable collection	<code>List.copyOf</code> , or an operation that answers the question directly
Wanting to reuse a class's code	Composition and delegation, not <code>extends</code>
A <code>switch</code> over a type that keeps gaining cases	A sealed hierarchy, so the compiler finds every switch
Many types, one operation over them	Polymorphism
Few types, many operations over them	Sealed types and pattern matching
Four constructors differing by parameter count	A builder
A method that may find nothing	Return <code>Optional</code> , not <code>null</code>
A method that may return no items	Return an empty collection, never <code>null</code>
Several <code>String</code> parameters in a row	Value objects, so a swapped argument fails to compile
A data carrier with no rules to protect	A record
An object whose attributes change but which stays the same thing	An entity, with equality on its identifier
Mutable static state	An injected instance, so it can be scoped, substituted and tested
An object used as a map key	Make it immutable, or it can become unreachable